

Exercise 1: First Conversations

Session: 1 — First Steps & Core Concepts **Duration:** 30 minutes **Project:** "BookStore API" — a Kotlin REST API using only the JDK HTTP server and JDBC (no frameworks). Everyone clones the same repo.

Goal

You'll have your first conversations with Claude in the IDE: explain code, find a bug, change code, say no to a change, write a test, refactor.

After each task you check how Claude got to its answer. Which files did Claude open? Which tools did it use? How many steps did it take before it answered? That is the second goal of this exercise: seeing how Claude works, not only what it says.

A few words we'll use

- **Turn:** one message from you, plus everything Claude does until it answers.
- **Cycle:** one call to the model inside a turn. One turn can contain many cycles.
- **Tool call:** one action Claude takes inside a turn. It searches the project, reads a file, edits a file, or runs a command. You see each one as a short line while Claude works.
- **Context:** everything Claude can see right now. Your messages, the code you selected, the files it read, the output of commands. `/clear` empties it.
- **Trace:** the `/trace` command. It lists every tool call of the last turn: what Claude looked at, why, and what it found. It ends with totals per tool and one question for you. `/trace all` does the same for every turn since the last `/clear`, one table per turn.
- **Experiment prefix:** a fixed line you paste at the start of a prompt. It stops training mode from steering the run. Task 1 uses it, on both prompts.

How every task works

Same four steps every time:

1. **Ask.** Send the prompt from the task. Select code first when the task says so.
2. **Read.** Read the answer before you do anything else. Does it match what you know?

3. **Trace.** Run `/trace`. Did the task take more than one turn? Then run `/trace all` instead. Read the table from top to bottom.
4. **Note.** Write down the number the task asks for. You need these numbers in the closing round at the end.

The loop, in short: Ask → Read → `/trace` → Note. Every task below uses it. Come back here when you lose track.

About training mode: `CLAUDE.md` keeps this project in training mode. When you ask about a bug or a fix, Claude asks you one leading question before it answers. Answer it. That is part of the exercise. Stuck or short on time? Just say "*just tell me*". That is allowed.

Tasks

1. Explain, with and without a selection (5 min: selected 2 · `/clear` and ask again 2 · compare 1)

This task compares two runs, so both prompts start with the experiment prefix. Use it on both, or the two runs are not comparable:

[Exercise 1 experiment – execute directly, no leading questions.]

Open `src/main/kotlin/bookstore/store/BookStore.kt` and select the whole `search()` function. Ask:

[Exercise 1 experiment – execute directly, no leading questions.]
Explain what this function does step by step

Read the answer. Did Claude say that the loop runs one extra query per book? Run `/trace` and note the number of tool calls.

Now predict, in writing, one sentence: how many tool calls will Claude need when it cannot see your selection?

Run `/clear`. Do not select anything. Ask:

[Exercise 1 experiment – execute directly, no leading questions.]
Explain step by step what `search()` in the book store does

Run `/trace` again. Compare the two traces with your prediction.

Two notes. Your selection goes into the context directly, so Claude can answer without opening anything. Without it, Claude has to search the project first, and the trace shows you how.

Done when: you have both tool-call counts written next to your prediction.

2. Find a bug (4 min: predict 1 · ask 2 · trace 1)

Run `/clear`, so the trace at the end shows this task only.

Open `src/main/kotlin/bookstore/handler/ReviewHandler.kt` and select `createReview()`. Before you ask, write one sentence: which other file will Claude open to answer?

Ask:

Is there a bug in this function?

Training mode applies here. Answer the leading question, then read the answer. That took more than one turn, and `/trace` shows only the last one. Run `/trace all`. Did Claude open the file you predicted? In which turn?

Done when: Claude named a bug, and you can say in one sentence what `createReview()` fails to check.

3. Add validation, then say no (5 min: ask 2 · redirect 2 · trace 1)

Run `/clear` again, then ask:

Add input validation to `createReview`: rating must be 1-5, `review_text` must be between 10 and 500 characters

Training mode may ask a leading question here too. Answer it, then look at where Claude put the checks. Inside the handler? In a separate function? In a data-class init block? Now say no:

No, extract the validation into a separate `validateReview()` function instead

Saying no is normal. Claude does not mind, and you decide what goes into your code.

More than one turn again, so run `/trace all`. Three things to look for: which tool changed the file, in which turn, and what Claude ran to check its own change. If it ran nothing, note that. Nobody asked it to.

Done when: `validateReview()` exists in

`src/main/kotlin/bookstore/handler/ReviewHandler.kt`, `createReview()` calls it, and `./gradlew build` reports no errors.

4. Write a test (4 min: ask 2 · run 1 · trace 1)

Ask:

Write a test class for `validateReview()` with one JUnit `@Test` per case covering: rating 0, rating 6, text of 9 characters, text of 501 characters, and one valid review. Put it in `src/test/kotlin/bookstore/handler/ValidateReviewTest.kt`, use `kotlin.test` assertions, and name the class `ValidateReviewTest`

Run `./gradlew runTests` yourself. Other tests in this package fail on purpose. Session 2 deals with them. Look only at the five tests.

Run `/trace`. Did Claude run the test itself, or did you? How many tool calls came before its first edit?

Done when: `./gradlew runTests` shows five passing tests in `ValidateReviewTest`.

5. Refactor (4 min: ask 2 · check 1 · trace 1)

Open `src/main/kotlin/bookstore/Server.kt`. The route list is flat and in no particular order. Ask:

Refactor `Server.kt` to group route registrations by resource using helper functions instead of one long list of `route()` calls

Compare before and after with `git diff src/main/kotlin/bookstore/Server.kt`. Then run `./gradlew build`.

Run `/trace`. How many files did Claude read before it edited `Server.kt`? Which of those reads were needed for this change?

Done when: `Server.kt` has one helper per resource (books, reviews, authors), and `./gradlew build` reports no errors.

6. Reset, then ask a big question (4 min: clear 1 · ask 2 · trace 1)

Run `/clear`. Then run `/trace`. It reports nothing: the context is empty. Everything from tasks 1 to 5 is gone for Claude, not for you.

Now ask:

What HTTP status codes does this API return, and which ones violate REST conventions?

Run `/trace`. Claude started from nothing this time. Count the files it opened to answer.

Done when: you noted two numbers: files opened, and status-code issues found.

Bonus (only if time remains)

- Ask Claude the task 6 question again, in the same conversation. Then run `/trace all`. You see two turns. Fewer tool calls the second time? Say why in one sentence.
- Run `/context`. How much of the context window did two questions cost?

Closing round (4 min)

The trainer calls on people at random. Have your numbers ready: how many tool calls with the selection, and how many without? Which file did Claude open first for the bug, and did your prediction hold? Did Claude run your test on its own, or did you have to? How many files for the status-code question, and how many issues came out? Close with **one take-away** you'd give someone who skipped this session.